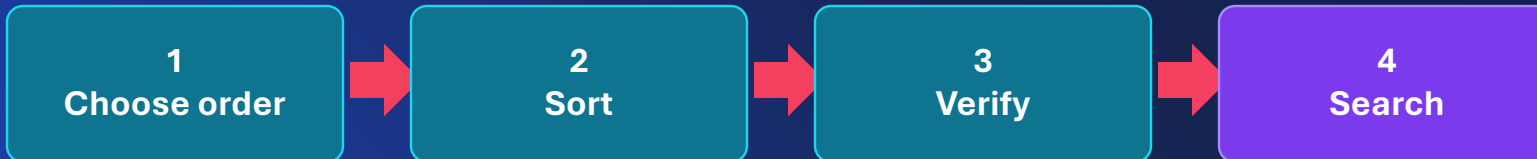




PROBLEM SOLVING USING PROGRAMMING (JAVA) / 26SC1101E

Library Sorting, Ordering Contracts and Binary Search

CO5 / MODULE 5 / WEEK 10 / LECTURE 4



SMART LOCKER: parcel reports and parcel-ID lookup

UNIFLOW: admission registers, merit order and student-ID lookup

Dr. N Chaitanya Kumar

Professor in CSE & Coordinator Freshman Engineering Department

Manual algorithms and library responsibility

CO3 explains algorithm behavior. CO5 applies trusted Java APIs to validated objects.

WHAT CO3 GAVE US

- Manual search and sort traces
- Comparison of adjacent or middle values
- Correctness reasoning for found and not-found cases
- Awareness that data order changes algorithm behavior

REUSE
REASONING

WHAT CO5 REQUIRES

- Choose the intended key and direction
- Supply a valid ordering contract for objects
- Call Arrays or Collections library methods
- Verify the order before binary search
- Interpret the returned position

BOUNDARY: We apply library methods. We do not reteach algorithm internals or Big-O analysis.

Sorting vocabulary and ordering choices



A precise ordering statement names the key, direction and any tie-breaker.

TERM	MEANING	SMART LOCKER	UNIFLOW
key	field used for comparison	parcel ID	student ID
ascending	smaller values appear first	PKG-101 before PKG-115	101 before 125
descending	larger values appear first	latest arrival first	9.2 CGPA before 8.7
natural order	class default order	parcel ID	student ID
alternate order	task-specific order	recipient or arrival time	name or CGPA
tie-breaker	second key for equal first keys	recipient, then arrival	CGPA, then student ID

Complete requirement: Sort students by CGPA descending. If CGPA is equal, use student ID ascending.

A comparator must return zero when the compared keys are equal.

Arrays.sort() with primitive values

The library rearranges the same array into ascending numerical order.

COMPLETE PROGRAM

```
import java.util.Arrays;

public class SortWeights {
    public static void main(String[] args) {
        double[] weights = {4.5, 1.2, 3.8, 2.0};
        System.out.println(Arrays.toString(weights));
        Arrays.sort(weights);
        System.out.println(Arrays.toString(weights));
    }
}
```

BEFORE SORT

[4.5, 1.2, 3.8, 2.0]

Arrays.sort

AFTER SORT

[1.2, 2.0, 3.8, 4.5]

VERIFY: Every value is less than or equal to the value on its right.

Arrays.sort() with String values

String natural order uses lexicographic comparison through compareTo().

UNIFLOW EXAMPLE

```
String[] departments = {"ECE", "CSE", "AI", "Civil"};  
System.out.println(Arrays.toString(departments));  
Arrays.sort(departments);  
System.out.println(Arrays.toString(departments));
```

POSITION	BEFORE	AFTER
0	ECE	AI
1	CSE	CSE
2	AI	Civil
3	Civil	ECE

CASE CAUTION

Uppercase and lowercase characters can produce an order that differs from a person's expectation. Normalize data when the requirement permits it, or supply a comparator designed for the required case behavior.

String order follows comparison rules. It does not understand academic or business meaning.

Natural ordering with Comparable<T>

A class implements one default comparison contract through compareTo().

Comparable<Parcel>

requires
compareTo(Parcel other)

IMPLEMENTS

class Parcel implements Comparable<Parcel>

NATURAL ORDER BY PARCEL ID

```
@Override
public int compareTo(Parcel other) {
    return this.parcelId.compareTo(other.parcelId);
}
```

RESULT	MEANING
negative	this object comes before other
zero	comparison keys are equal
positive	this object comes after other

Do not promise only -1, 0 or 1. The contract requires negative, zero or positive.

Smart Locker natural order by parcel ID



Parcel implements Comparable so Arrays.sort() can order a Parcel array directly.

PARCEL COMPARISON

```
class Parcel implements Comparable<Parcel> {
    String parcelId;
    String recipient;

    Parcel(String parcelId, String recipient) {
        this.parcelId = parcelId;
        this.recipient = recipient;
    }

    public int compareTo(Parcel other) {
        return parcelId.compareTo(other.parcelId);
    }

    public String toString() { return parcelId; }
}
```

CALL AND OUTPUT

```
Parcel[] parcels = {
    new Parcel("PKG-115", "Ravi"),
    new Parcel("PKG-101", "Anitha"),
    new Parcel("PKG-108", "Meera")
};
Arrays.sort(parcels);
System.out.println(Arrays.toString(parcels));
```

OUTPUT

```
[PKG-101, PKG-108, PKG-115]
```

Natural-order contract: parcel ID ascending for every direct Parcel sort.

UniFlow natural order by student ID



Student ID supplies a predictable default register order.

STUDENT COMPARISON

```
class Student implements Comparable<Student> {
    int studentId;
    String name;
    double cgpa;

    Student(int id, String name, double cgpa) {
        this.studentId = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    public int compareTo(Student other) {
        return Integer.compare(studentId,
            other.studentId);
    }
}
```

BEFORE	COMPARE KEY	AFTER
125, Kavya	125	101, Asha
101, Asha	101	113, Zoya
113, Zoya	113	125, Kavya

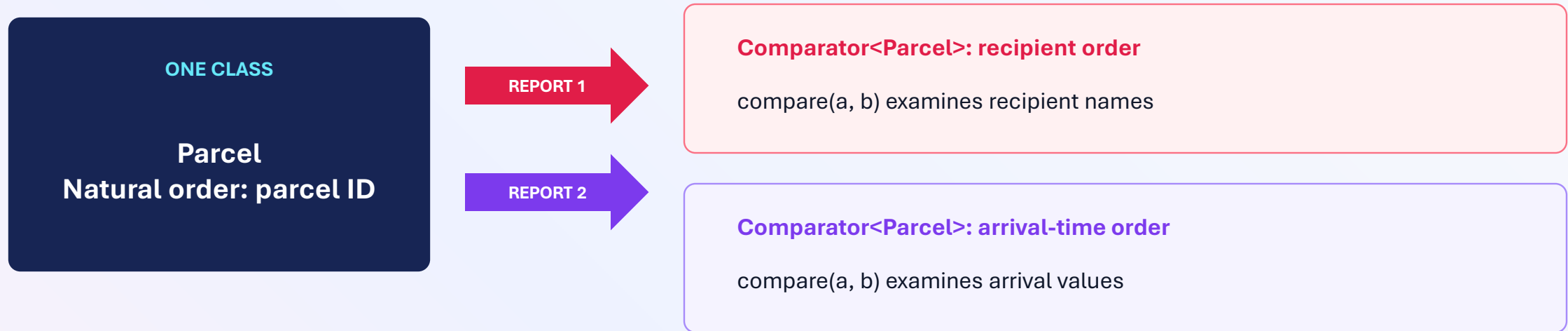
WHY Integer.compare?

It clearly compares two int values and returns the required negative, zero or positive result.

UniFlow natural order: student ID ascending. Name and CGPA remain alternate orders.

Alternate ordering with Comparator<T>

A comparator supplies a task-specific order without changing the class's natural order.



METHOD	OWNER	PURPOSE
<code>compareTo(other)</code>	the domain class	one natural order
<code>compare(a, b)</code>	a Comparator object	one alternate order

Comparable answers the default-order question. Comparator answers a report-specific question.

Smart Locker alternate parcel reports



Recipient order uses arrival time as a tie-breaker when two recipients match.

COMPARATOR IMPLEMENTATION

```
class RecipientThenArrival implements Comparator<Parcel> {  
    public int compare(Parcel a, Parcel b) {  
        int byRecipient =  
a.recipient.compareToIgnoreCase(b.recipient);  
        if (byRecipient != 0) return byRecipient;  
        return a.arrivalTime.compareTo(b.arrivalTime);  
    }  
}
```

```
Arrays.sort(parcels, new RecipientThenArrival());
```

PARCEL	RECIPIENT	ARRIVAL	FINAL POSITION
PKG-104	Anitha	10:30	2
PKG-101	Anitha	09:10	1
PKG-108	Meera	08:45	3
PKG-115	Ravi	11:20	4

TIE-BREAK TRACE

Anitha equals Anitha on the first key. The program then compares 09:10 and 10:30, so PKG-101 appears first.

The second comparison runs only when the first comparison returns zero.

UniFlow alphabetical and merit orders



Different reports reuse the same Student objects with different Comparator implementations.

NAME ORDER

```
class ByName implements Comparator<Student> {  
    public int compare(Student a, Student b) {  
        return a.name.compareToIgnoreCase(b.name);  
    }  
}
```

DESCENDING CGPA WITH ID TIE-BREAK

```
class ByCgpaDescending implements Comparator<Student> {  
    public int compare(Student a, Student b) {  
        int byCgpa = Double.compare(b.cgpa, a.cgpa);  
        if (byCgpa != 0) return byCgpa;  
        return Integer.compare(a.studentId, b.studentId);  
    }  
}
```

STUDENT	CGPA	MERIT POSITION
113, Zoya	9.20	1
101, Asha	8.70	2
125, Kavya	8.70	3

WHY b.cgpa BEFORE a.cgpa?

Double.compare(b.cgpa, a.cgpa) reverses the usual ascending direction. Larger CGPA values move earlier. Equal CGPAs use student ID ascending.

Descending order reverses the operands. The tie-breaker keeps equal-CGPA results predictable.

Comparable and Comparator selection guide



The required report decides which comparison contract the sort call should use.

QUESTION	COMPARABLE	COMPARATOR
Where is the rule written?	inside the domain class	inside a separate comparator class
Method	compareTo(other)	compare(a, b)
How many roles?	one natural order	multiple alternate orders
Smart Locker example	parcel ID	recipient or arrival time
UniFlow example	student ID	name or descending CGPA
Sort call	Arrays.sort(items)	Arrays.sort(items, comparator)

COMMON MISTAKES

Returning only 1 or 0, reversing some comparisons but not others, changing the comparison key during one sort, or forgetting zero for equal keys.

Selection rule: use natural order for the class default. Supply Comparator for a task-specific report.

Collections.sort() and List.sort() usage



A provided List can use the same natural and alternate ordering contracts.

PROVIDED LIST EXAMPLES

```
List<Student> students = getStudents();

Collections.sort(students);
// natural student-ID order

students.sort(new ByName());
// alternate alphabetical order

Collections.sort(students, new ByGpaDescending());
// alternate merit order
```

DATA	NATURAL ORDER	ALTERNATE ORDER
array	Arrays.sort(a)	Arrays.sort(a, c)
provided List	Collections.sort(list)	list.sort(c) or Collections.sort(list, c)

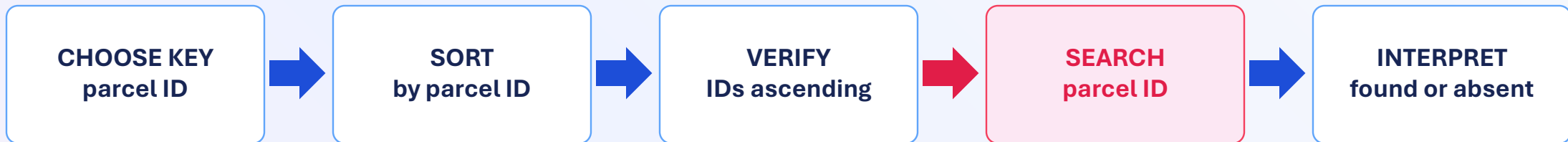
SCOPE FOR THIS LECTURE

The List is already available. We use its sorting APIs. CO6 will explain collection creation, selection and design in detail.

The ordering contract remains the same even when the container API changes.

Binary-search ordering precondition

Binary search is correct only when sorting and searching use the same key and comparison rule.



CORRECT CONTRACT

Sorted by parcel ID
Searched by parcel ID
Result can be trusted

MISMATCHED CONTRACT

Sorted by recipient
Searched by parcel ID
Result cannot be trusted

Binary search does not inspect whether the data is correctly sorted. The programmer must guarantee it.

Arrays.binarySearch() result trace



A nonnegative result identifies a matching index. A negative result means the key is absent.

SMART LOCKER PROGRAM

```
String[] ids = {"PKG-101", "PKG-104", "PKG-108", "PKG-115"};  
Arrays.sort(ids);
```

```
int found = Arrays.binarySearch(ids, "PKG-108");  
int absent = Arrays.binarySearch(ids, "PKG-109");
```

```
System.out.println(found);  
System.out.println(absent);
```

OUTPUT

```
2  
-4
```

KEY	PRECONDITION	RETURN	INTERPRETATION
PKG-108	IDs ascending	2	found at index 2
PKG-109	IDs ascending	-4, negative	not found

INDEX STRIP

index	0	1	2	3
value	PKG-101	PKG-104	PKG-108	PKG-115

For this course, interpret any negative return as not found. Insertion-point encoding remains outside scope.

Object binary search and mismatched ordering



The search object and stored objects must use the same comparison key as the prior sort.

CORRECT STUDENT-ID SEARCH

```
Collections.sort(students);  
// ordered IDs: [101, 113, 125]  
  
Student key = new Student(113, "", 0.0);  
int pos = Collections.binarySearch(students, key);  
  
System.out.println(pos >= 0 ?  
    "Found at " + pos : "Not found");  
  
OUTPUT  
Found at 1
```

STEP	CORRECT CASE	WRONG CASE
sort	student ID	student name
search	student ID	student ID
match?	yes	no
trust result?	yes	no

BTL-4 DIAGNOSIS

Repair the wrong case by sorting naturally by student ID before the student-ID search, or use a search comparison that matches name order.

A plausible output from a mismatched search remains unreliable because the required precondition is false.

Guided dual-application ordering exercise



Choose the API and comparison contract before predicting the final order or search result.

CASE	DATA	REQUIREMENT	YOUR DECISION
A	Parcel[]	parcel ID ascending	API and contract
B	Parcel[]	recipient, then arrival	Comparator logic
C	Student[]	CGPA descending, ID tie-break	Comparator logic
D	List<Student>	student ID search	sort and search calls
E	Parcel[] sorted by recipient	parcel ID binary search	diagnose and repair

ANSWER KEY

A: Arrays.sort with Comparable. B and C: Arrays.sort with a Comparator. D: natural List sort followed by Collections.binarySearch. E: resort by parcel ID before searching.

Evidence: name the key, direction, tie-breaker, API and binary-search precondition.

Thank you



Lecture 4 exit evidence and final verification

SUBMIT ONE COMPLETE TRACE

1. State the required key and direction
2. Select Comparable or Comparator
3. Write the library sort call
4. Verify the resulting order
5. Apply one matching binary search
6. Explain one mismatched-order failure

FINAL QUESTIONS

What defines natural order?
When is Comparator required?
Why must equal keys return zero?
What must happen before binary search?
Why is a negative result useful?

NEXT: LECTURE 5

Paths, Files, buffered input/output and persistence across separate program runs

Dr. N Chaitanya Kumar

Professor in CSE & Coordinator Freshman Engineering Department

LECTURE 4 COMPLETE: Choose order, sort, verify, then search with the same contract.